FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Iago Amorim Lebron de Lima

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Quadro de entregas 2
4 Atividades realizadas 3
4.1 Atividade 1 — Agenda telefônica em Flask 3
4.2 Atividade 2 — Catálogo de filmes — Tom Hanks 4
4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação 6
4.4 Atividade 4 — Controle de acesso por papel — RBAC 8
4.5 Atividade 5 — Logs e auditoria 9
4.6 Atividade 6 — Upload e perfil de usuário 11
5 Considerações finais 12
6 Declaração de autoria 12

1 Introdução

O presente relatório descreve as atividades práticas desenvolvidas ao longo do bimestre na disciplina de Introdução à Computação em Nuvem. O fio condutor da disciplina foi a construção e evolução de um projeto de "Catálogo de Filmes", focado na filmografia do ator Tom Hanks, projetado desde o início para vivenciar os desafios de uma arquitetura moderna e escalável.

Inicialmente, o projeto foi concebido de maneira simplificada, mas rapidamente evoluiu para uma arquitetura baseada em microsserviços. Durante esse processo de transformação, a aplicação passou a ser orquestrada através do Docker, segregando responsabilidades em serviços distintos para autenticação, catálogo e logs. Funcionalidades avançadas foram incorporadas gradualmente, como Controle de Acesso Baseado em Papéis (RBAC), auditoria de eventos utilizando Redis Streams e armazenamento de arquivos de perfil (avatares) na nuvem utilizando o Garage.

O objetivo deste relatório é apresentar de forma técnica e transparente as escolhas arquiteturais realizadas em cada uma das etapas do projeto, demonstrando o funcionamento de cada integração. Nas seções a seguir, cada atividade entregue é detalhada juntamente com suas respectivas evidências de execução e links para o repositório onde o código se encontra versionado.

2 Metodologia

Todas as atividades foram desenvolvidas de forma contínua no mesmo repositório público do GitHub, integrando um único projeto de catálogo de filmes. O ambiente de desenvolvimento e execução foi orquestrado utilizando Docker e Docker Compose, englobando a stack técnica de Python com FastAPI e Flask, bancos de dados relacionais (MariaDB), cache e mensageria (Redis) e object storage (Garage).

Para assegurar a veracidade das entregas, as datas e horas exatas foram extraídas diretamente do histórico de commits do repositório (`git log`). Os links apresentados nas fichas de atividades apontam para os respectivos commits que representam a conclusão de cada etapa. Os prints registrados demonstram o sistema em pleno funcionamento, além dos artefatos de versionamento exigidos.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	07/08/2026	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 18:17	entregue
3	Desacoplando o login — microsserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	28/08/2026 11:19	entregue

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	01/09/2026 18:04	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	30/09/2026 16:25	entregue com atraso
6	Upload e perfil de usuário	Página de perfil com avatar armazenado no Garage (S3); só a referência fica no banco relacional.	02/10/2026	30/09/2026 21:07	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	07/08/2026
Situação	entregue
Evidência	Docker Hub — imagem + print da interface
Link	https://hub.docker.com/r/lebron057/minha-api-flask

O que foi feito. Foi desenvolvido um projeto simples de uma agenda telefônica utilizando Flask e Jinja para renderização do frontend. A API possui rotas para listar e cadastrar novos registros (com nome, telefone e valor), persistindo as informações em um arquivo JSON local (`dados.json`). A aplicação foi containerizada utilizando uma imagem leve do Python (`python:3.10-slim`) e a imagem final foi publicada no Docker Hub.

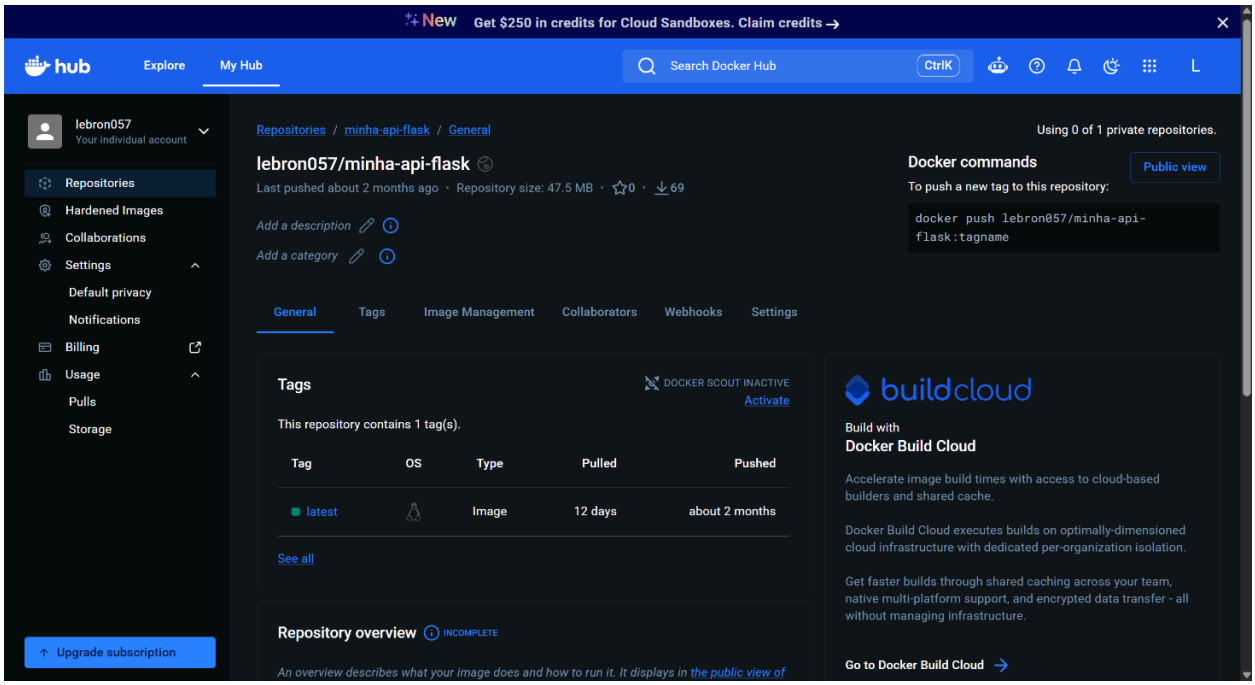


Figura 1: Atividade 1 — evidência da entrega (link, data e hora visíveis)

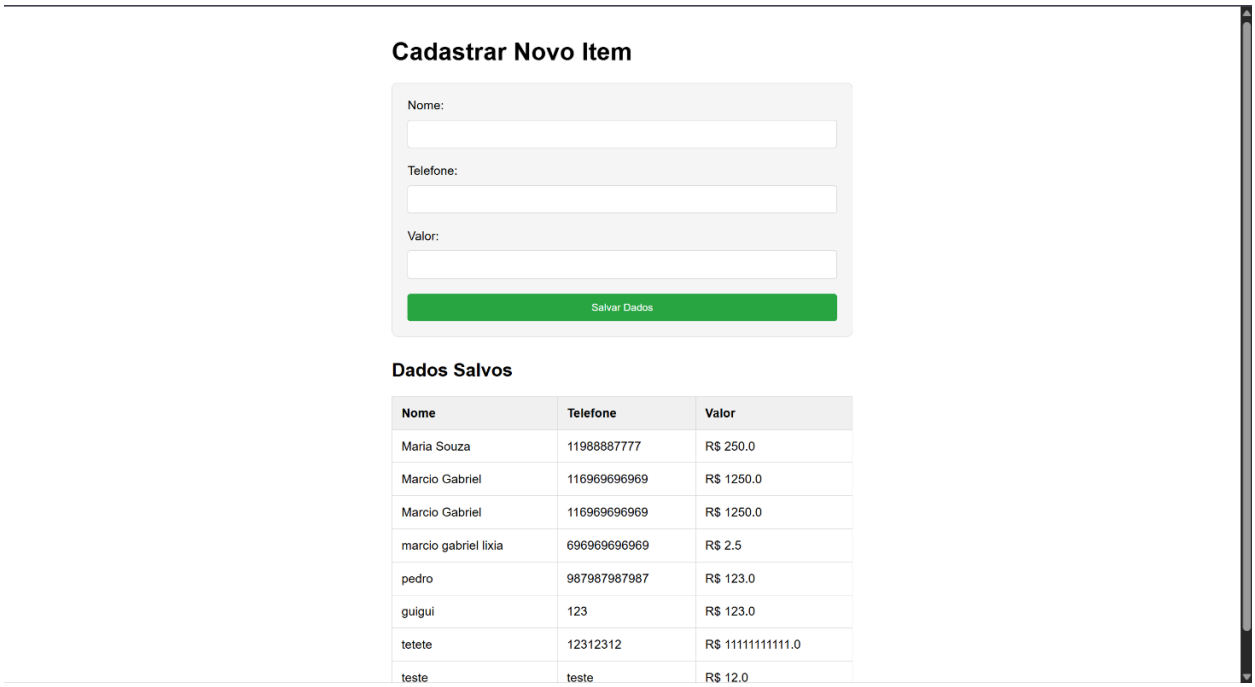


Figura 2: Atividade 1 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. A implementação do código ocorreu de forma tranquila. A maior dificuldade encontrada foi construir o `Dockerfile`, pois foi a minha primeira vez criando um. Isso foi resolvido estudando as instruções básicas (como `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE` e `CMD`) para configurar adequadamente o ambiente e garantir que o Flask respondesse corretamente dentro do contêiner.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026

Data realizada	20/08/2026 18:17
Situação	entregue
Evidência	GitHub — commit + README + print do catálogo
Link	https://github.com/Lebron057/tom-hanks-avestruz/commit/85774b5f66b1a30ccda9b077e6c285f025ab2ab7

O que foi feito. Foi construída a primeira versão do catálogo de filmes do Tom Hanks utilizando FastAPI e Jinja2. Realizei a integração com a API do TMDB para buscar dados e posters de filmes, e estruturei o banco de dados relacional MariaDB para persistir os usuários e relacioná-los com seus filmes favoritados (segregação por usuário). A arquitetura base já previa a orquestração com Docker e Docker Compose, definindo os serviços da aplicação web e do banco de dados na mesma rede, simplificando os testes locais e preparando terreno para os futuros microsserviços.

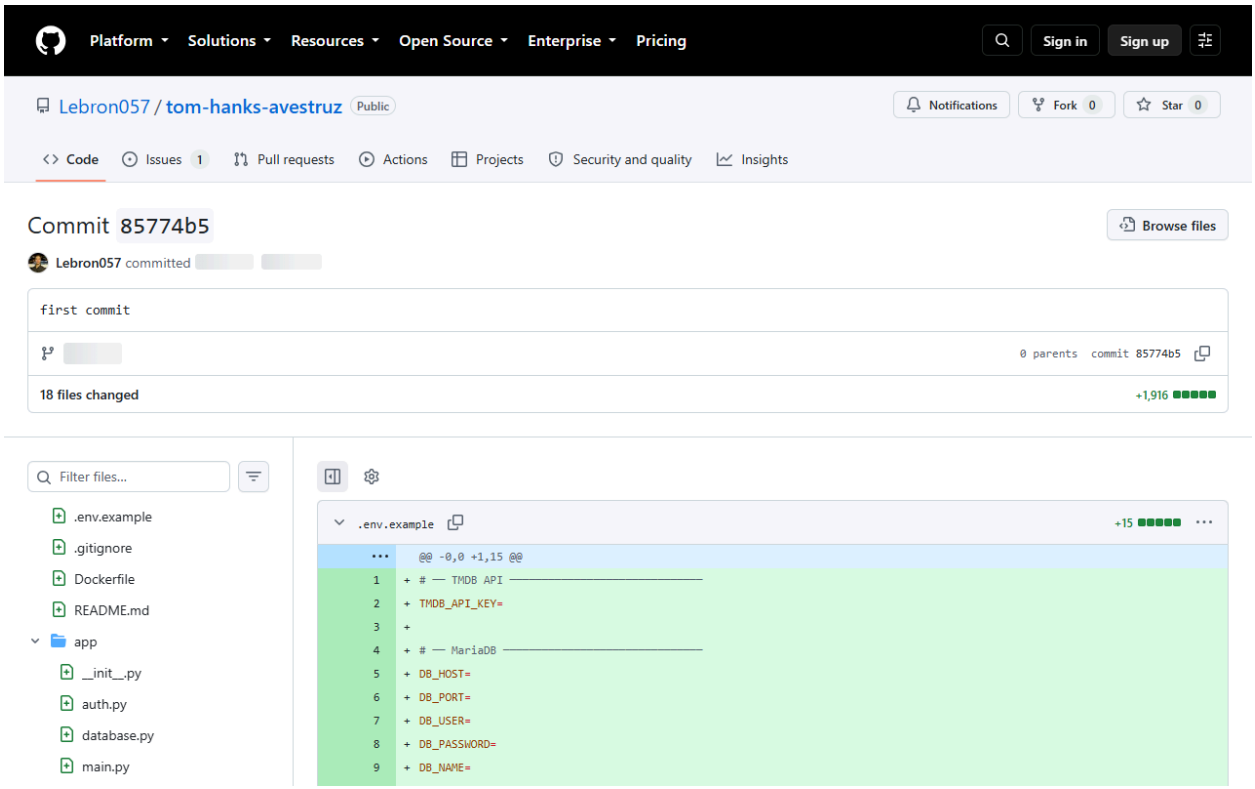


Figura 3: Atividade 2 — evidência da entrega (link, data e hora visíveis)

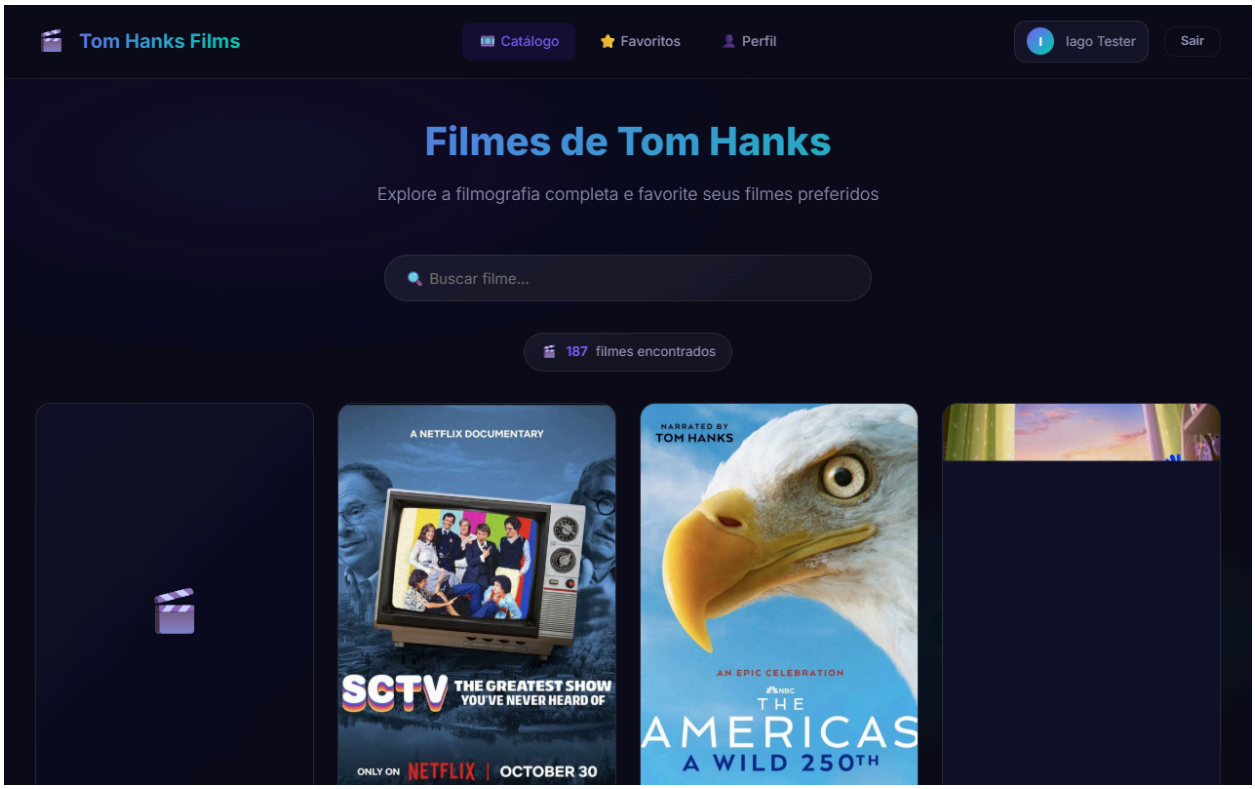


Figura 4: Atividade 2 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. A principal dificuldade foi entender a estrutura de resposta da API do TMDB e como isolar corretamente o container do MariaDB para garantir a persistência dos dados entre os reinícios dos containers. Isso foi resolvido criando volumes mapeados no `docker-compose.yml`.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	28/08/2026 11:19
Situação	entregue
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando
Link	https://github.com/Lebron057/tom-hanks-avestruz/commit/31b256bc52118cf9ca047c6b927ee49ebc6653eb

O que foi feito. O fluxo de autenticação foi completamente extraído do monólito e isolado num novo microsserviço (`auth-service`). Este serviço, acessível apenas na rede interna do Docker, tornou-se responsável por gerenciar a tabela de usuários, cadastrar, validar credenciais via JWT e oferecer a funcionalidade de recuperação de senha integrada ao Mailtrap. O serviço principal (`catalog-service`) passou a atuar como um cliente, repassando as requisições de login e registro para o `auth-service` via HTTP interno, desacoplando a responsabilidade e aprimorando a segurança geral do sistema.

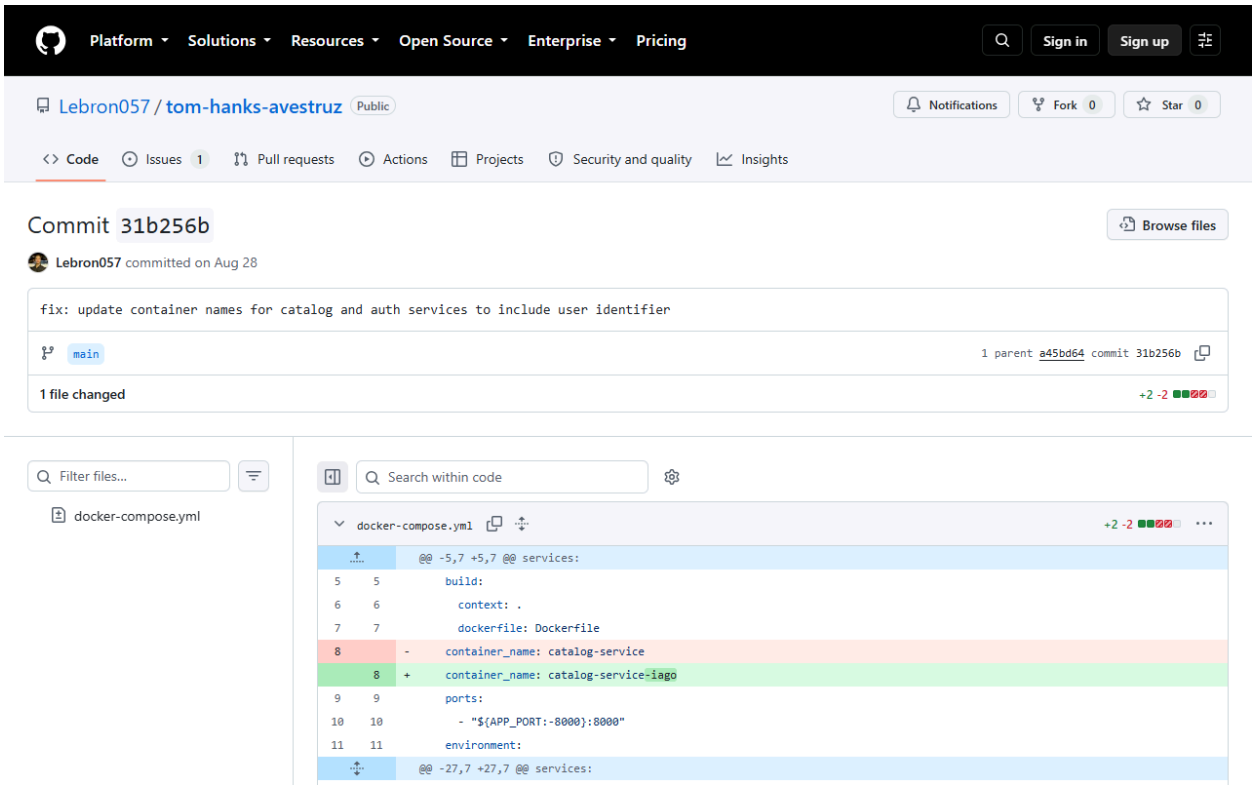


Figura 5: Atividade 3 — evidência da entrega (link, data e hora visíveis)

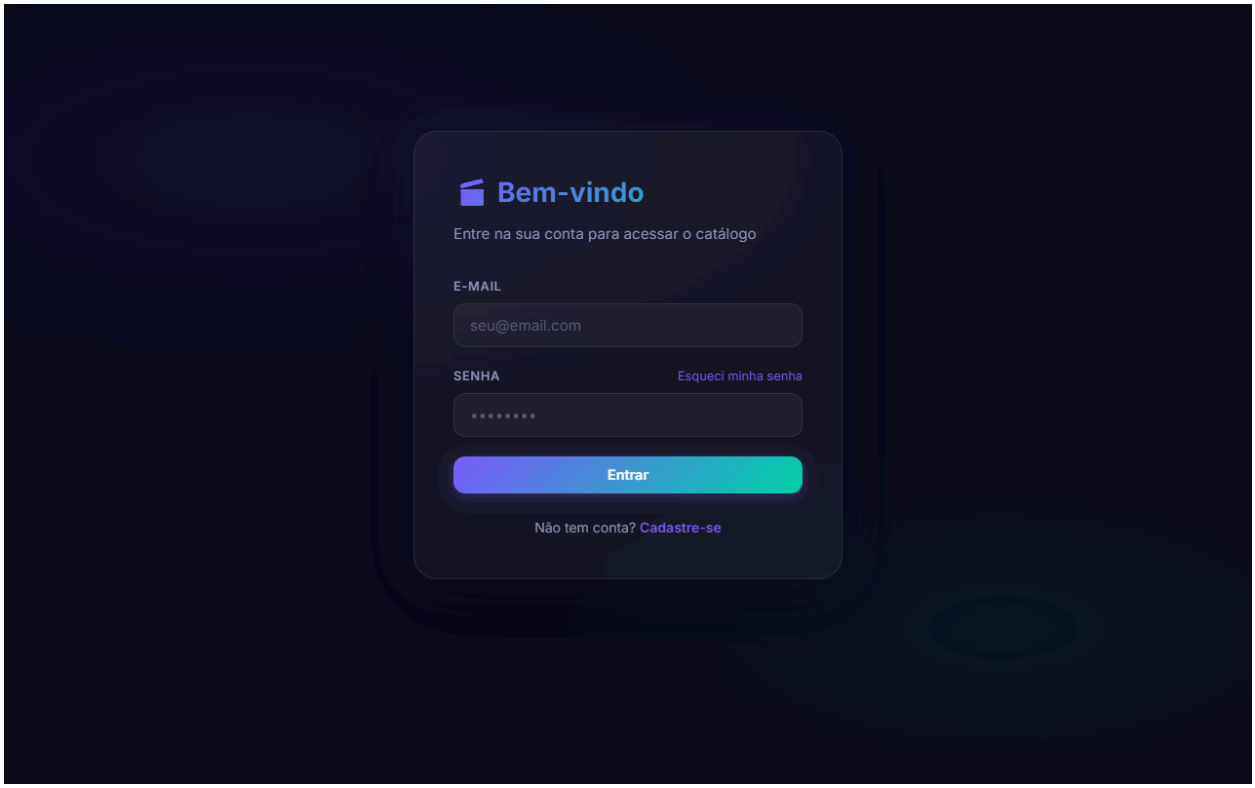


Figura 6: Atividade 3 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. Uma dificuldade notável foi gerenciar os cookies (como `access_token`) a partir das respostas do novo microsserviço de volta para o cliente final passando pelo serviço de catálogo. Isso exigiu atenção extra no repasse correto de headers e cookies de autenticação pelo `catalog-service`.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	01/09/2026 18:04
Situação	entregue
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/Lebron057/tom-hanks-avestruz/commit/a3c1a000c7089e5c7554084497f02d21b2154485

O que foi feito. Foi implementado o Controle de Acesso Baseado em Papéis (RBAC). Utilizei a abordagem de Enforcement Centralizado (Padrão A), onde a validação e autorização ocorrem diretamente no serviço de borda (`catalog-service`). Este serviço lê as permissões e decide se a ação pode prosseguir (ex: apagar um comentário) dependendo do campo `role` presente no JWT, gerado pelo `auth-service`. A lógica adicionada garante que um usuário comum só pode apagar os seus próprios comentários, ao passo que administradores podem excluir qualquer comentário em nome da moderação. Se um usuário não autorizado tenta a operação, o backend devolve adequadamente um erro 403.

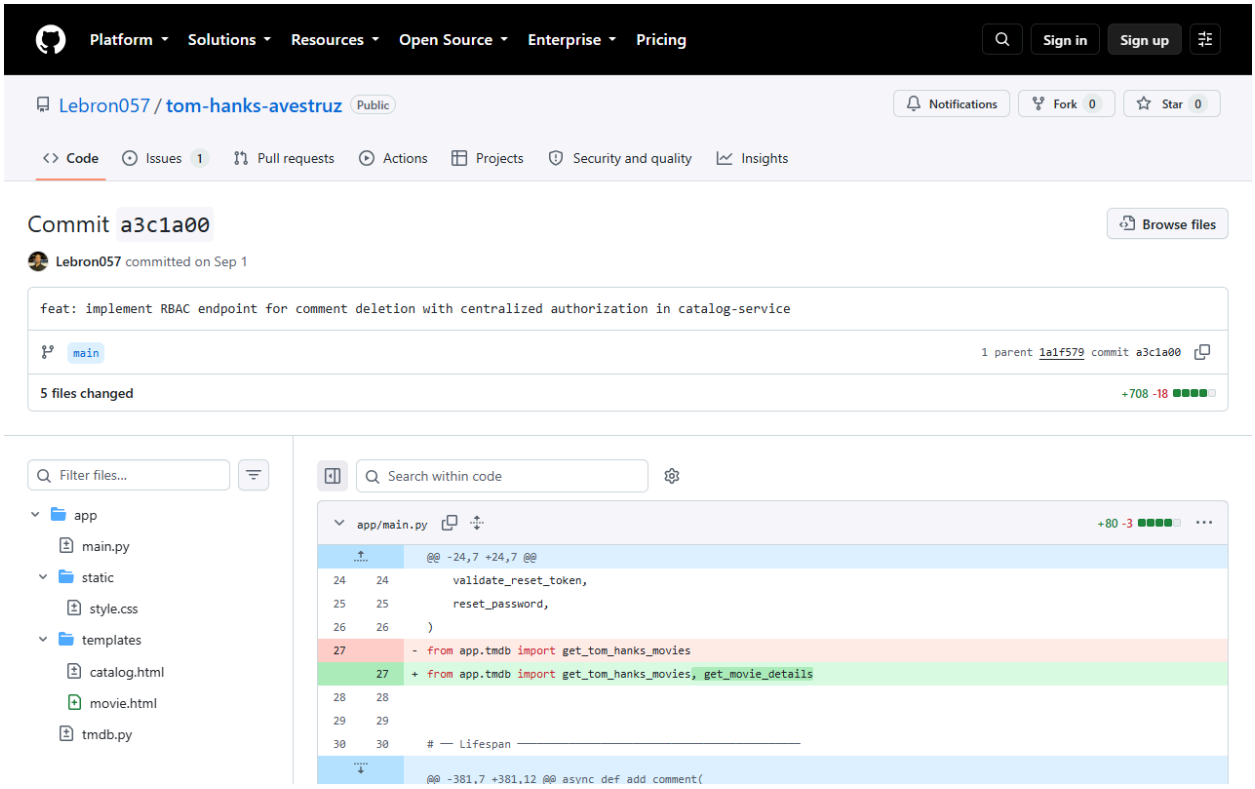


Figura 7: Atividade 4 — evidência da entrega (link, data e hora visíveis)

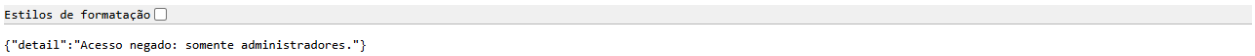


Figura 8: Atividade 4 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. A dificuldade foi projetar adequadamente a checagem no endpoint e garantir a correta transmissão do papel (role) de administrador dentro do payload JWT do microsserviço de autenticação, resolvido mapeando o campo corretamente durante a geração do token.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	30/09/2026 16:25
Situação	entregue com atraso
Evidência	GitHub — commit + print da consulta de logs pelo admin
Link	https://github.com/Lebron057/tom-hanks-avestruz/commit/1a2c29eb97532bce3aa2f1261d6412985578062e

O que foi feito. Foi construído um terceiro microsserviço (`log-service`) dedicado à gravação e leitura de trilhas de auditoria das ações do sistema, mantendo a chamada de log como não-bloqueante ("best-effort"). Diversos pontos críticos do backend foram mapeados para emitir eventos de log via HTTP interno. Para o armazenamento, foi escolhido utilizar o Redis Streams (XADD) em oposição ao Redis List, porque as Streams fornecem naturalmente IDs únicos com timestamps cronológicos nativos, ideal para séries temporais e log de auditoria distribuída. O administrador também conta com um endpoint seguro para visualizar as entradas reversamente.

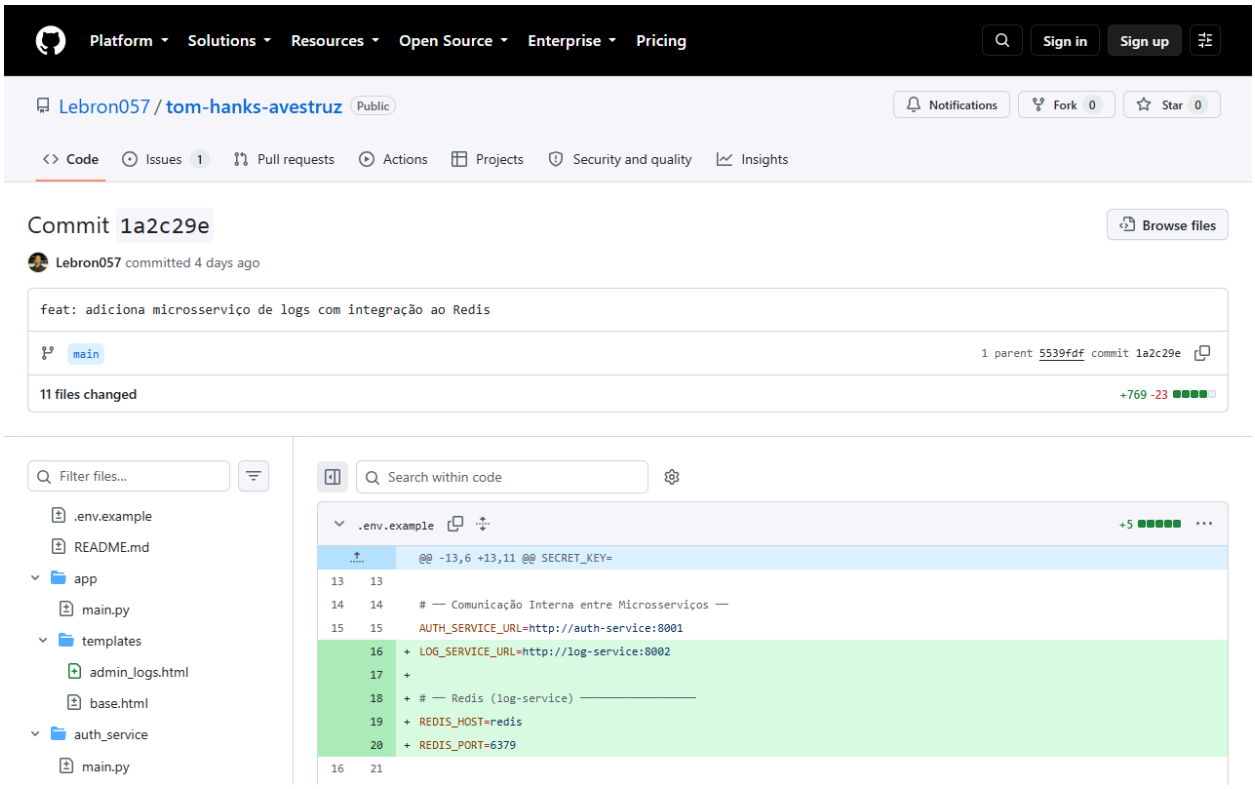


Figura 9: Atividade 5 — evidência da entrega (link, data e hora visíveis)

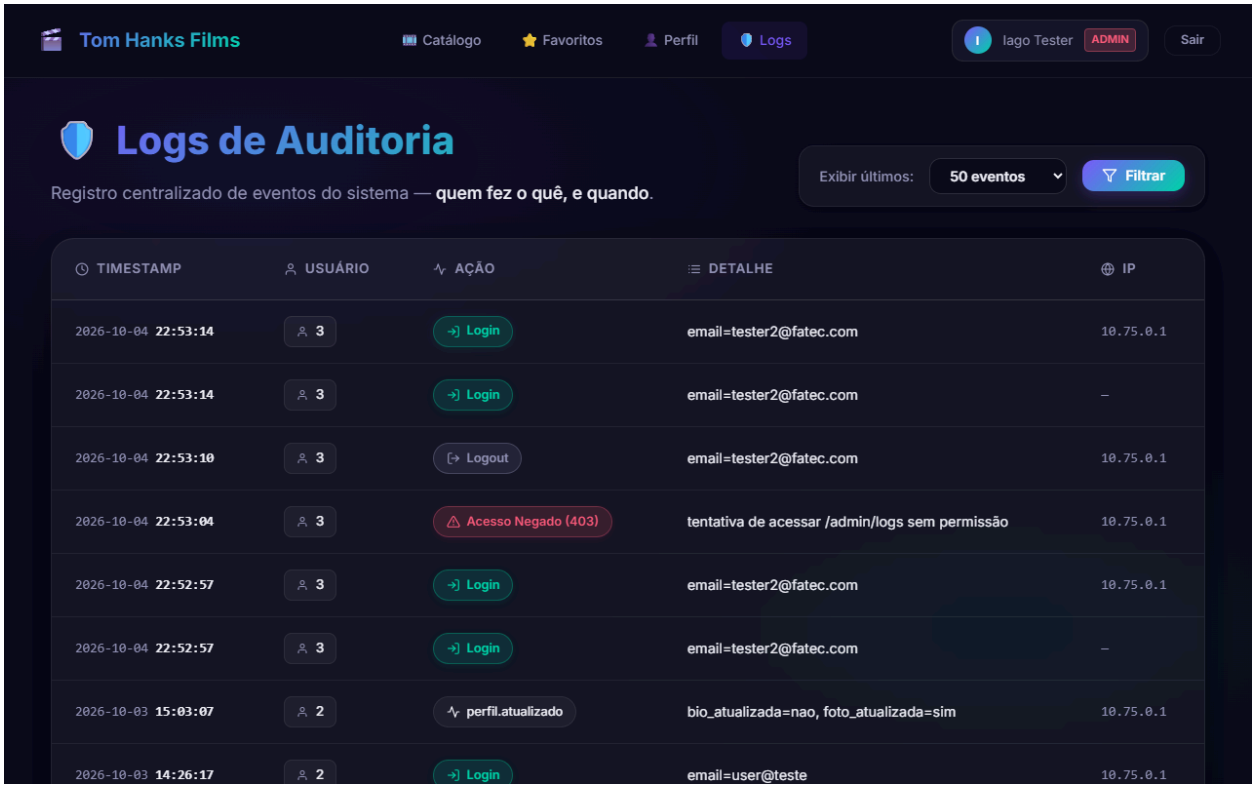


Figura 10: Atividade 5 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. Foi um desafio configurar o CORS e integrar a interface administrativa com o log-service, pois exigia requisições HTTP enviando corretamente os tokens via Bearer Auth no header, o que foi resolvido repassando a chamada backend-to-backend via `httpx` no serviço de catálogo.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar armazenado no Garage (S3); só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	30/09/2026 21:07
Situação	entregue
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/Lebron057/tom-hanks-avestruz/commit/500ef8d21393fb8eef2d7851a2829de8eb507034

O que foi feito. Desenvolvi a página de perfil público e a funcionalidade de upload de foto de perfil (avatar). Para evitar o armazenamento de binários pesados no banco de dados relacional, utilizei o Garage (object storage compatível com S3) hospedado via contêiner. No MariaDB armazenou-se apenas o caminho/identificador textual da foto gerada, mantendo o banco ágil. Em relação ao gerenciamento dos objetos, optei por criar o bucket com política de leitura pública (Opção A). Foi a melhor estratégia para simplificar e aliviar o gargalo de processamento das requisições na aplicação, já que avatares são essencialmente dados de visualização abertos dentro da plataforma e não exigem URLs pré-assinadas dinâmicas restritas.

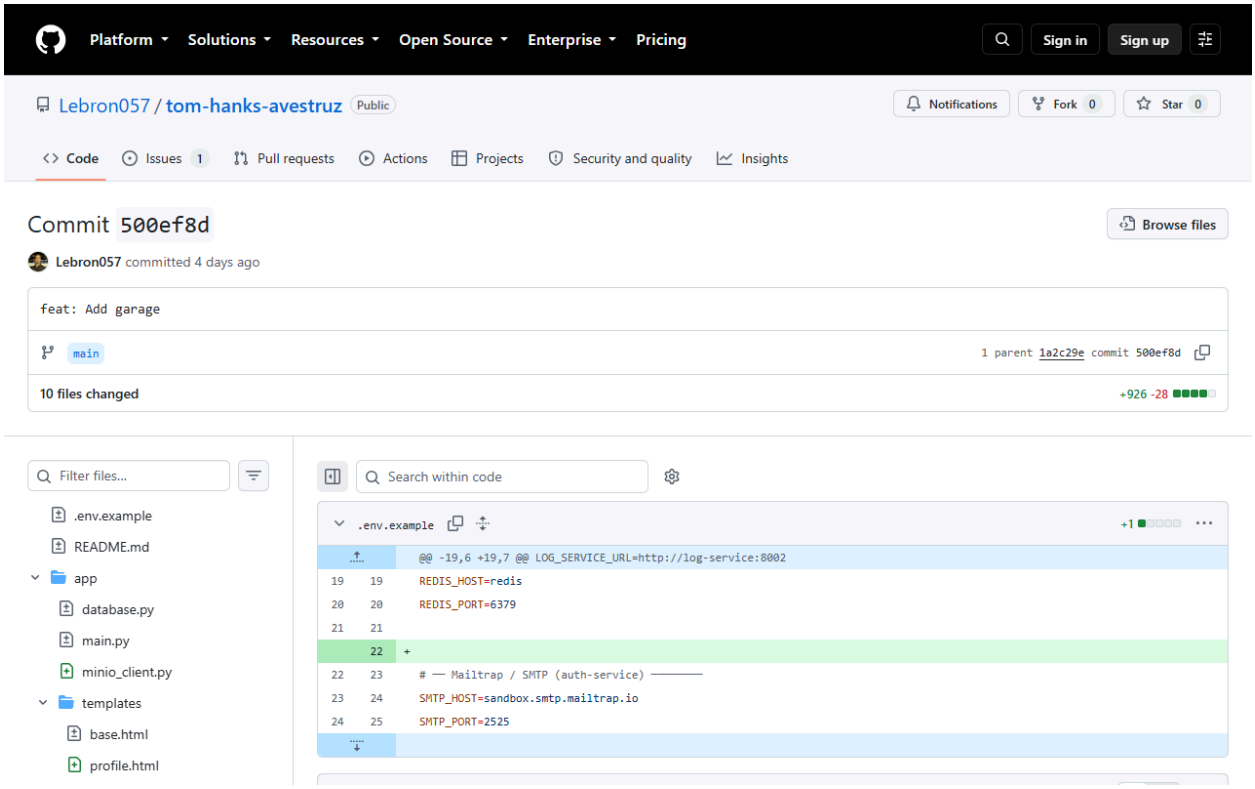


Figura 11: Atividade 6 — evidência da entrega (link, data e hora visíveis)

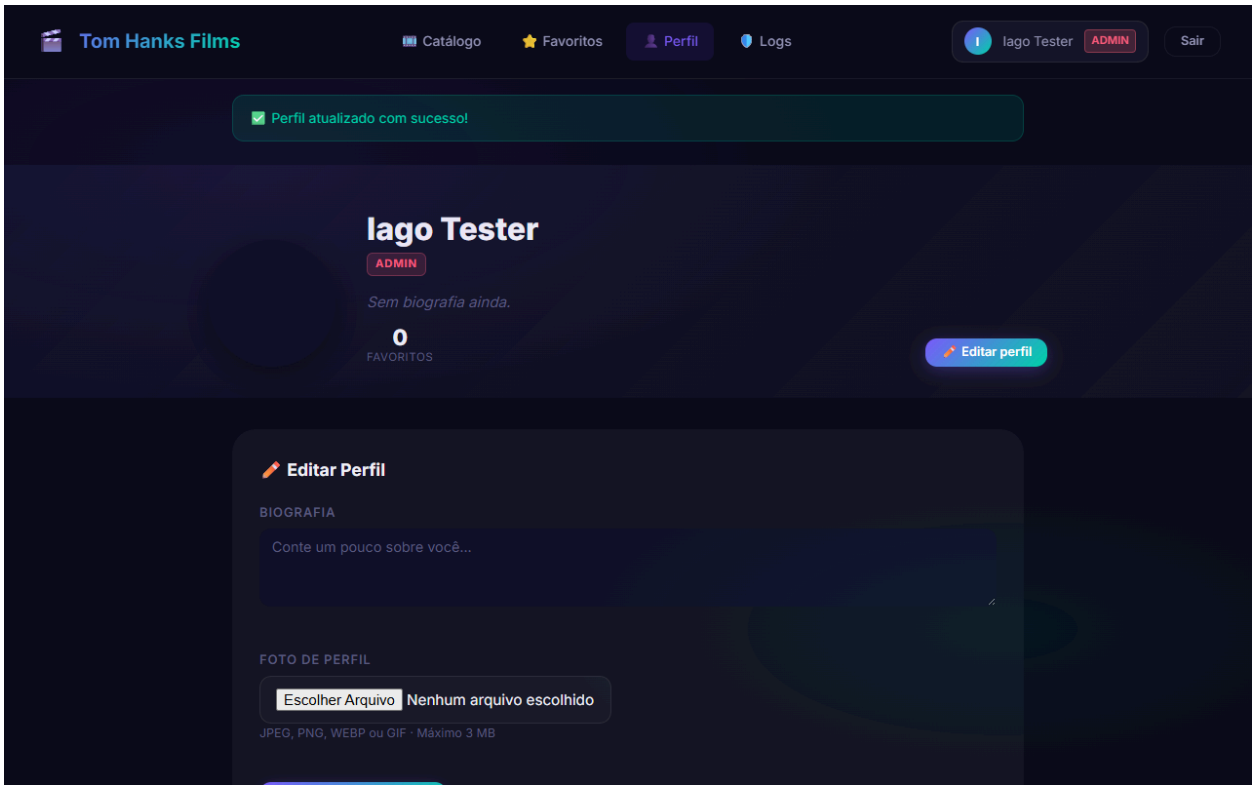


Figura 12: Atividade 6 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. A conexão entre a biblioteca cliente S3 em Python e a estrutura do Garage gerou alguns erros na criação inicial do bucket e definição da política de permissões, que foram devidamente tratados no código adicionando retentativas e configurações adequadas de roteamento no path-style.

5 Considerações finais

O bimestre foi marcado por uma profunda transição conceitual, guiando a implementação de uma arquitetura centralizada (monolítica) para a orquestração distribuída via Docker. Foi imensamente enriquecedor aprender na prática sobre a descentralização do fluxo de autenticação e a necessidade vital da observabilidade através dos eventos de logs, e por fim o armazenamento estático usando object storage.

O maior desafio, indiscutivelmente, residiu na comunicação e propagação segura de tokens (JWT e RBAC) entre contêineres e o manuseio adequado das sessões, assegurando que as regras de negócio fossem aplicadas sempre pelo ponto correto (Padrão A). Para o futuro, pretendo me aprofundar na automatização dos deploys (CI/CD) para garantir a estabilidade sempre que um novo componente for adicionado à rede.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

gov.br
Documento assinado digitalmente
IAGO AMORIM LEBRON DE LIMA
Data: 07/10/2026 20:00:02-0300
Verifique em https://validar.iti.gov.br

Iago Amorim Lebron de Lima Pompeia, 07/10/2026